

KAPITOLA 4: TRANSPORTNÍ VRSTVA MODELU OSI

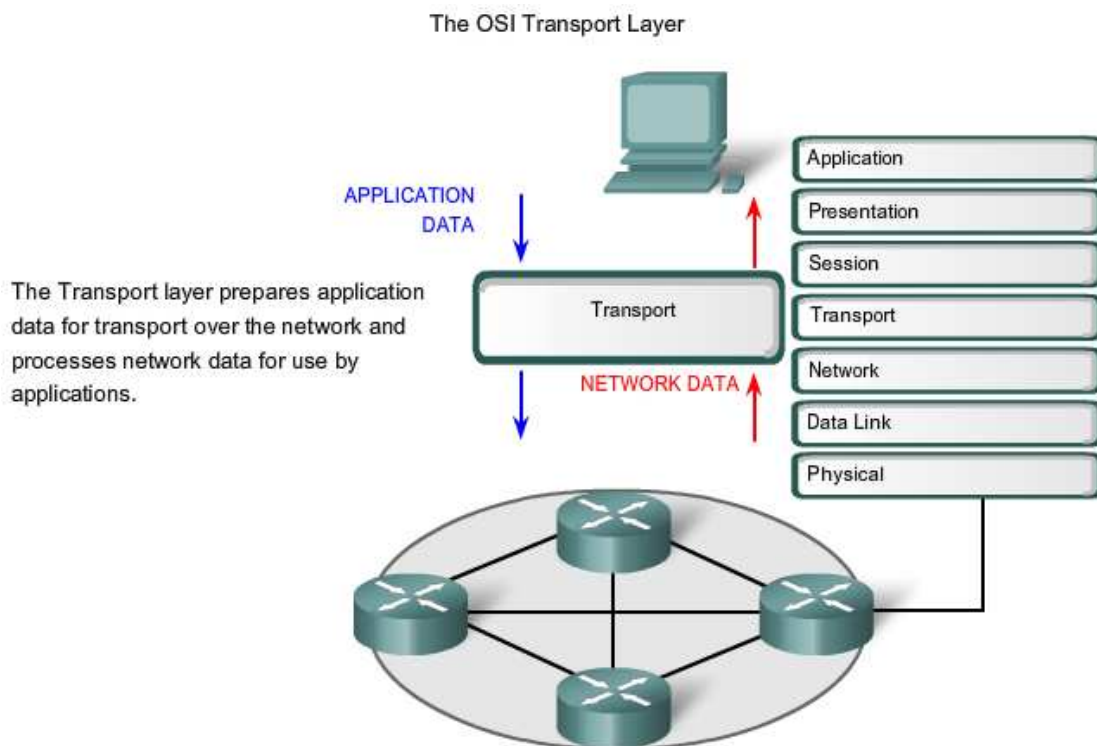
4.0 Úvod

Na jednom zařízení (PC) mohou lidé současně využívat více služeb – e-mail, webové stránky,... Data z každé aplikace jsou paketizována (zabalena do balíčku), přeneseny a doručena příslušné aplikaci na cílovém zařízení (PC nebo Server).

Úkolem transportní vrstvy je přijmout data z Aplikační vrstvy a připravit je na přenos přes síť – TRANSPORTNÍ VRSTVA REALIZUJE END – TO – END PŘENOS APLIKAČNÍCH DAT.

Transportní vrstva plní tyto funkce:

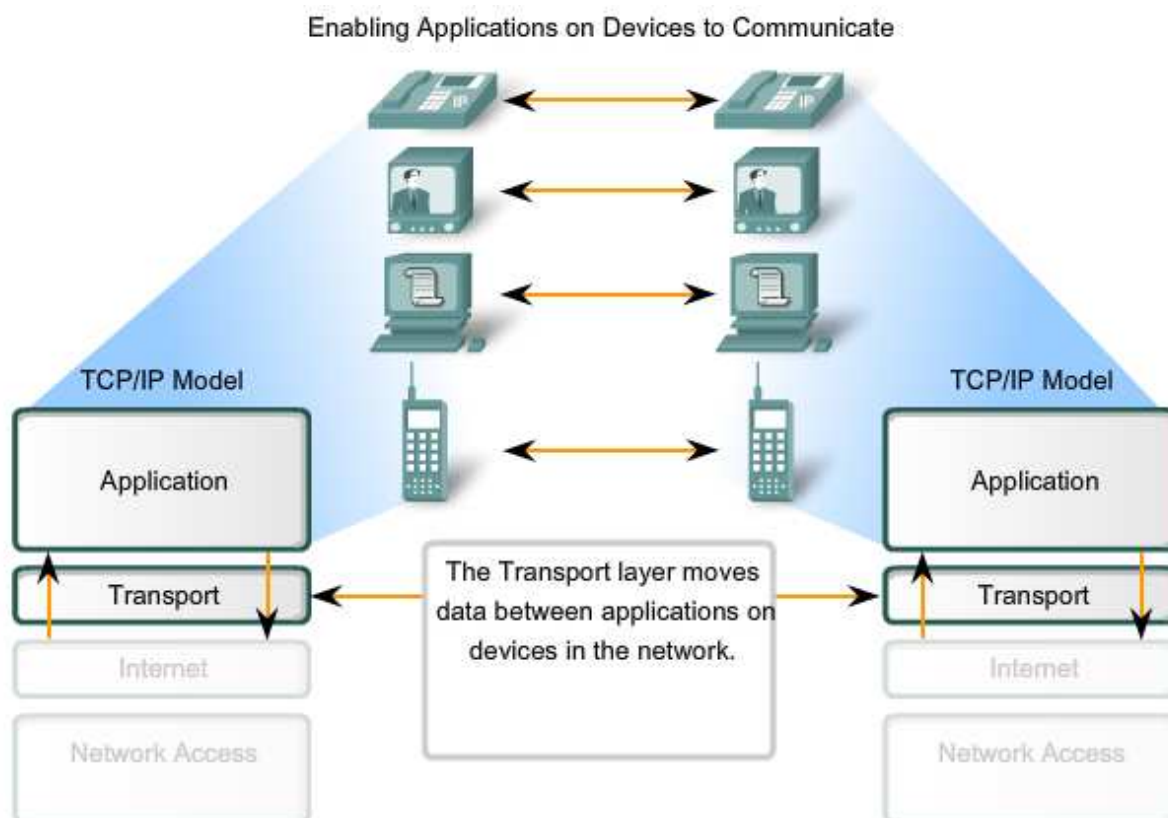
- **segmentaci dat** – dělení aplikačních dat na menší části a na přijímací straně opětovné složení (reassembly) dat ze segmentů
- **umožňuje používání více aplikací v jednom časovém okamžiku**
- **zajišťuje doručení dat ve správném pořadí příslušné aplikaci**
- **detekuje chyby a zajišťuje jejich korekci** – při nedoručení části zprávy



4.1.1 Smysl transportní vrstvy

Mezi hlavní činnosti transportní vrstvy patří:

- **sledování (dohled) a udržování jednotlivých komunikací mezi aplikacemi** – jeden uživatel může současně používat několik aplikací pro komunikaci – úkolem transportní vrstvy (T.V.) je udržovat jednotlivé komunikace
- **segmentace dat a řízení každého segmentu** – aplikační data jsou v T.V. dělena na menší části, které jsou opatřeny hlavičkou, která udává příslušnost k dané komunikaci
- **znovu složení (reassembly) segmentů** – do původních aplikačních dat na přijímací straně – na základě informací v hlavičce transportní vrstvy se provádí znovu složení dat a předání dat do příslušné aplikace (podle portu)
- **identifikace různých aplikací** – na základě portů, které jsou přiřazeny jednotlivým aplikacím



Data mají různé požadavky

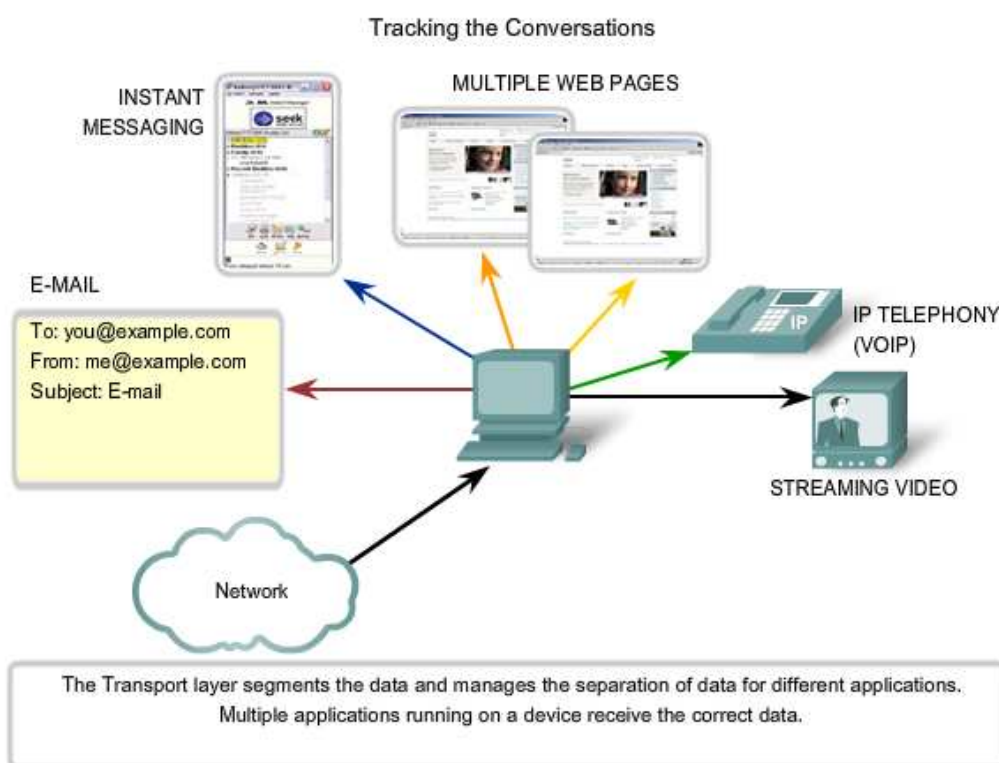
Na základě rozdílných požadavků aplikací existuje více transportních protokolů. Některé aplikace vyžadují přenesení veškerých dat a jejich předání do aplikační vrstvy ve správném pořadí, jiné aplikace tolerují částečnou ztrátu dat v průběhu přenosu přes síť.

Na základě výše zmíněných požadavků aplikačních protokolů se transportní protokoly dělí na protokoly, které:

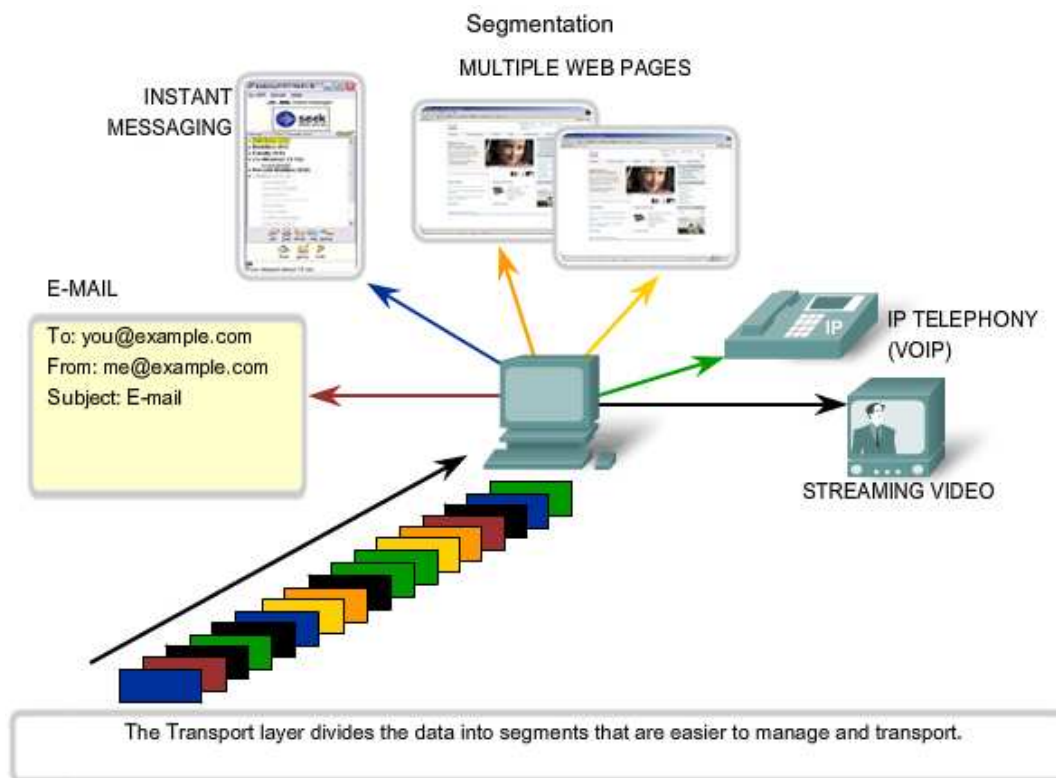
- poskytují pouze základní funkce pro doručení jednotlivých datových částí mezi aplikacemi – nepoužívají přídavné procesy, které zvětšují množství přenášených řídicích informací v hlavičce transportní vrstvy ⇒ **výsledkem je rychlé doručení dat bez zajištění spolehlivosti (reliability) doručení všech částí odeslaných dat** – tyto protokoly jsou výhodné pro aplikace vyžadující malé zpoždění (delay) při přenosu např. IP telefonie, live video,... **Tyto vlastnosti má protokol UDP (User Datagram Protocol).**
- doplňují přenášená data o větší množství řídicích informací v hlavičce transportní vrstvy – **čímž se zajišťuje spolehlivé doručení všech částí odeslané zprávy** – tyto protokoly jsou používány aplikacemi e-mail, webovými prohlížeči, ... **Tyto vlastnosti má protokol TCP (Transmission Control Protocol).**

Oddělení více komunikací

Úkolem transportní vrstvy odesílatele je přijímat data od různých aplikací, dělit je na menší části, doplnit je nezbytnými řídicími informacemi (hlavičkou) a odeslat do sítě. Transportní vrstva příjemce pak obdržené segmenty předává na základě informací z hlavičky příslušné aplikaci.



Používáním segmentace dat v transportní vrstvě lze zajistit používání více aplikací v jednom okamžiku – data od těchto aplikací se multiplexují (sdružují) – jsou prokládána (interleaved) s ostatními segmenty od jiných aplikací. Na základě čísla portu jsou pak segmenty u příjemce doručovány transportní vrstvou do patřičné aplikace.



Každá datová komunikace mezi aplikačními vrstvami realizována ve formě přenosu segmentů se nazývá KONVERZACE.

Bez použití segmentace by šlo v jednom časovém okamžiku používat pouze jednu aplikaci, která by po celou dobu používání obsazovala datovou linku.

4.1.2 Kontrola konverzací

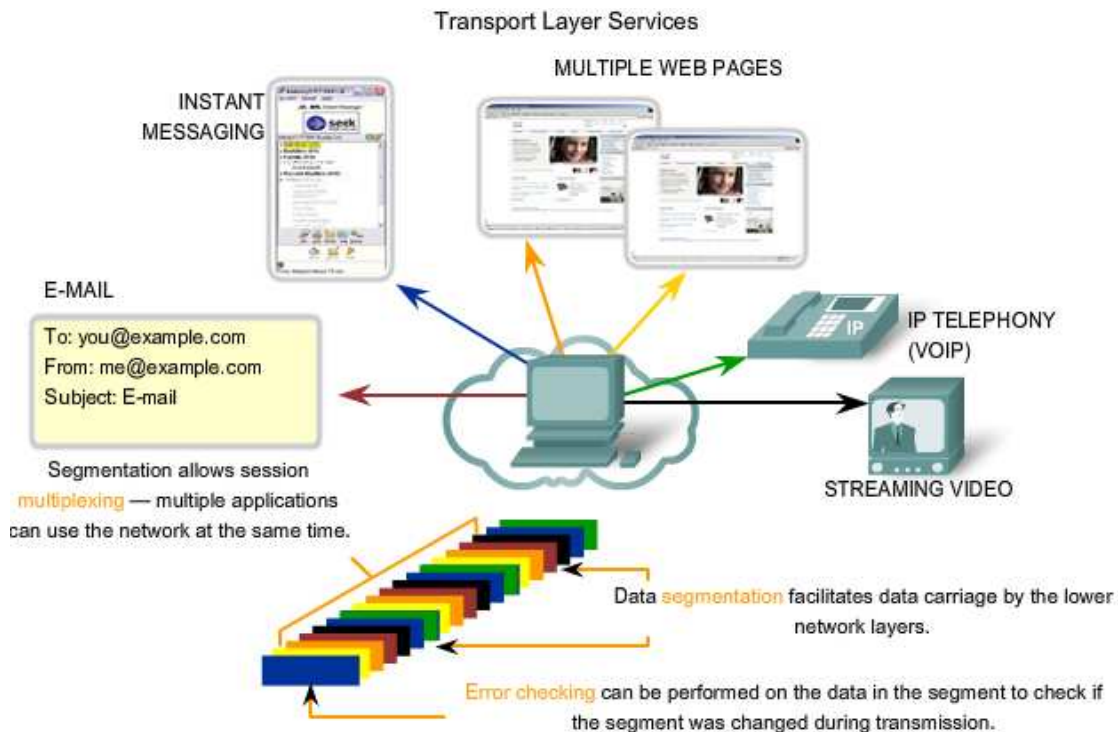
Dvě hlavní funkce jsou společné pro všechny transportní protokoly:

- **segmentace a znovu poskládání dat (reassembly)** – datové sítě mají omezenou velikost dat obsažených v jedné PDU – proto se musí realizovat segmentace
- **multiplexování konverzací** – možnost současného využívání více aplikací – data od aplikací jsou označovány porty a vzájemně prokládány na společnou linku.

Některé transportní protokoly mohou navíc – na základě informací obsažených v hlavičce – realizovat:

- spojově orientovaný přenos (connection – oriented)

- spolehlivé doručování (reliable delivery)
- doručení dat ve správném pořadí (ordered data reconstruction)
- řízení přenosu (flow control)



Vytvoření spojení (Establishing Session)

Transportní vrstva může poskytovat spojově orientovaný přenos dat mezi aplikacemi ⇒ před samotným přenosem dat se zřídí spojení mezi transportními vrstvami a do takto vytvořeného spojení se vkládají jednotlivé segmenty ⇒ lze tak lépe řídit přenos segmentů.

Spolehlivé doručení (Reliable Delivery)

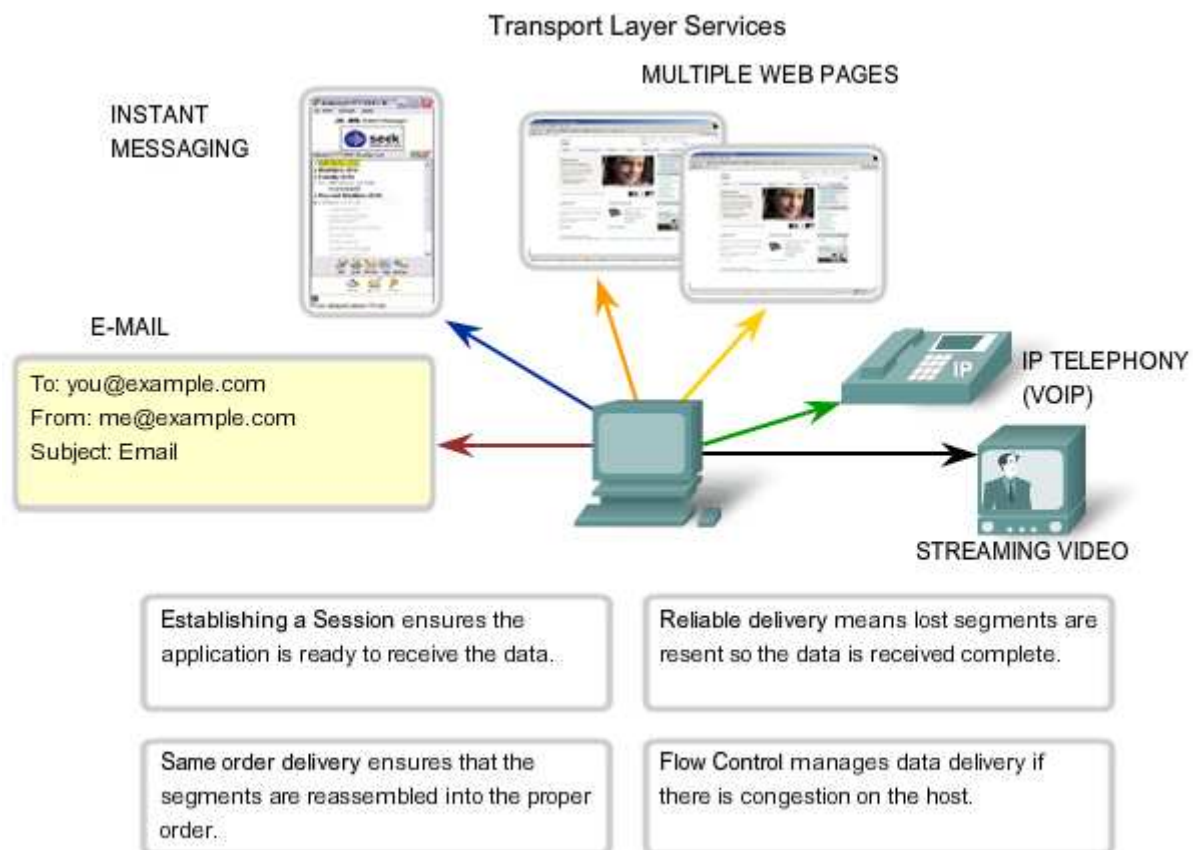
V průběhu přenosu jednotlivých částí dat přes síť může dojít k poškození (corrupt) nebo úplné ztrátě (loss) dat – transportní vrstva může zajišťovat znovu odeslání (retransmit) poškozených nebo ztracených částí dat.

Doručení ve správném pořadí (Same Order Delivery)

Jednotlivé části dat mohou putovat přes síť různými cestami ⇒ jednotlivé části dat dorazí do cílového PC v nesprávném pořadí. Pomocí číslování odeslaných segmentů seřadí transportní vrstva příjemce segmenty do správného pořadí a předá je do příslušné aplikace.

Řízení přenosu (Flow Control)

PC mají limitované velikosti paměti a rychlost zpracování dat $\Rightarrow$ mohlo by dojít k přetížení (overtaxed) PC – proto existuje možnost regulace množství odeslaných dat. Řízení přenosu slouží jako prevence před ztrátou segmentů a nutností jejich znovu odeslání.



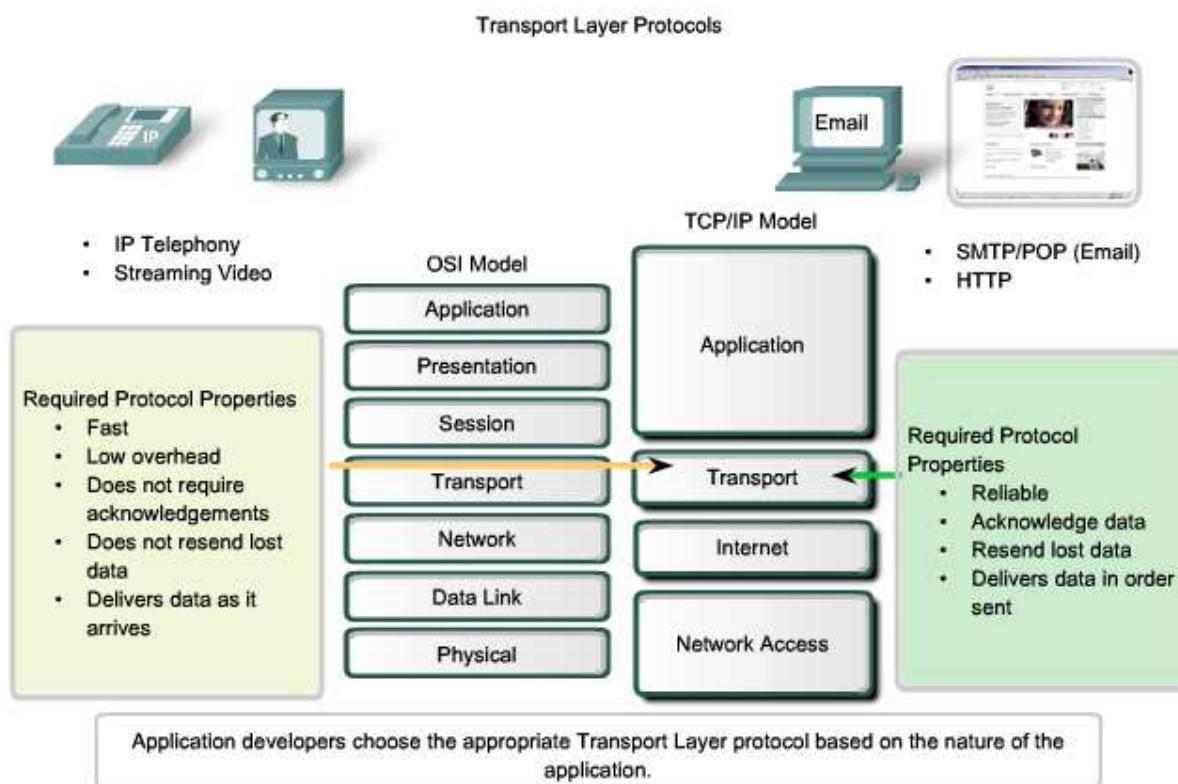
4.1.3 Podpora spolehlivé komunikace

Různé aplikace mají odlišné požadavky na přenos dat přes síť $\Rightarrow$ proto existuje více transportních protokolů, které tyto požadavky plní.

Transportní protokoly mohou podporovat spolehlivé doručení dat – tím je myšleno, že veškeré odeslané data (části dat) budou doručeny do cílového PC. Pro zajištění tohoto požadavku musí transportní vrstva vykonávat tři činnosti:

- sleduje přenášená data
- potvrzuje přijatá data
- znovu odesílá nepotvrzená data

Pro zajištění spolehlivého doručení dat musí hlavička transportní vrstvy obsahovat více informací než je tomu v případě nespolehlivého (best effort) doručování.



Tvůrci aplikací se musí na základě vlastností dané aplikace rozhodnout, zda je zapotřebí spolehlivé nebo nespolehlivé doručení dat pro tuto aplikaci.

Nespolehlivé doručování znamená, že příjemce odesilateli nepotvrzuje přijetí dat a současně není zajištěno, že veškerá data budou přenesena a předána do aplikace ve správném pořadí.

4.1.4 TCP a UDP

Oba protokoly řídí komunikaci a umožňují používat více aplikací současně – přesto každý z nich poskytuje specifické funkce.

UDP (User Datagram Protocol)

Má následující vlastnosti:

- jednoduchý, nespojově orientovaný protokol (connectionless protocol) specifikovaný v doporučení RFC 768
- výhodou je malá hlavička – 8 Byte

- datové celky se nazývají **DATAGRAMY** a jsou posílány metodou **BEST EFFORT** – což znamená s maximálním úsilím (snahou) bez garance doručení
- protokol UDP používají aplikační protokoly DNS, DHCP a služby IP telefonie a live video

UDP Datagram



TCP (Transmission Control Protocol)

Má následující vlastnosti:

- je to spojově orientovaný protokol (connection-oriented protocol) specifikovaný v doporučení RFC 793
- má větší hlavičku – 20 Byte $\Rightarrow$ čímž oproti protokolu UDP umožňuje garantovat doručení veškerých dat ve správném pořadí
- datový celky se nazývá **SEGMENT**
- protokol TCP se používá u webových prohlížečů (protokol HTTP), přenosu souborů (protokol FTP), služby e-mail (protokoly SMTP, POP),

TCP Segment



4.1.5 Adresace porty

Pro označení a rozlišení současně probíhajících aplikací na jednom PC používá transportní vrstva **PORTY**, které jsou obsaženy v hlavičce TCP i UDP protokolu – **zdrojový (source) a cílový (destination) port**.

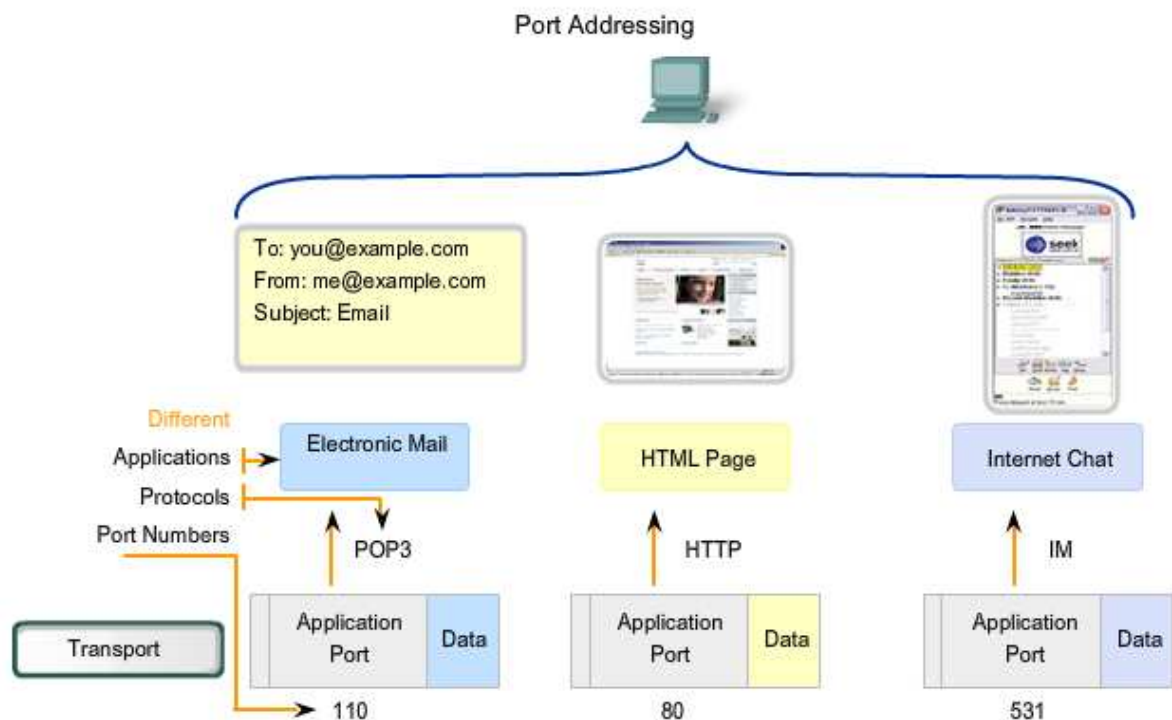
Číslo portu je 16-bitové číslo v rozsahu $0 \div 65535$.

Porty mohou být:

- **pevně přiděleny** – platí pro cílové porty u serveru – např. na portu 80 naslouchá webový server – obecně jsou to porty v rozsahu $0 \div 1023$
- **generovány** – platí pro zdrojové porty používané PC – generování provádí operační systém – tyto porty jsou > 1023 ($1024 \div 65535$)

Když klient odesílá dotaz na server, tak musí znát cílový port, na kterém daný server naslouchá na dotazy. Jakmile server odpovídá na dotaz klienta, tak jako cílový port použije zdrojový port obsažený v dotazu. Po obdržení odpovědi transportní vrstva příjemce na základě cílového portu předá data do patřičné aplikace.

PORT + IP ADRESA = SOCKET – tyto dva údaje jednoznačně identifikují aplikaci a PC, na kterém tato aplikace běží. **SOCKET se zapisuje ve tvaru IP ADRESA : PORT** například 192.168.0.1:80.



Data for different applications is directed to the correct application because each application has a unique port number.

Přidělování portů provádí organizace **IANA** (Internet Assigned Numbers Authority). **Porty se dělí na:**

a) Dobře známé (Well Known)

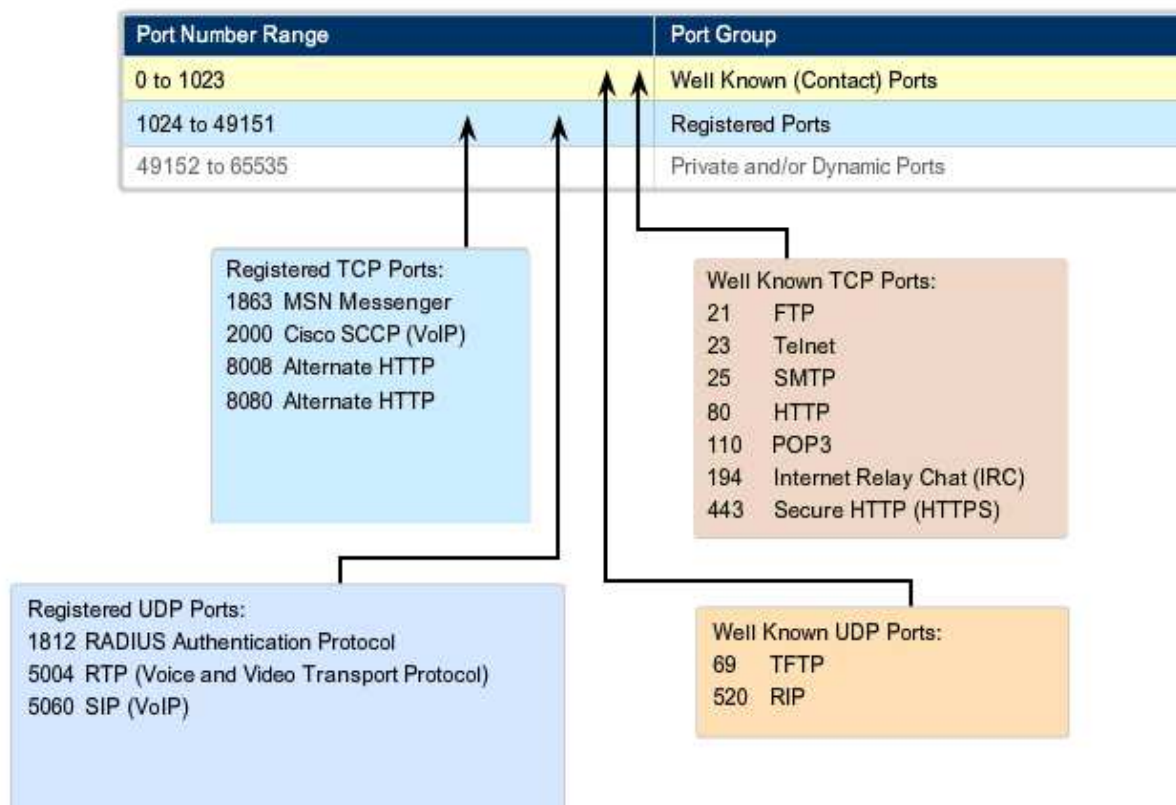
- rozsah od **0 ÷ 1023**
- jsou určeny pro označení serverových služeb a aplikací – např. HTTP port 80
- na těchto portech naslouchají servery na dotazy klientů

b) Registrované (Registered)

- rozsah od **1024 ÷ 49151**
- jsou určeny pro označení aplikací a služeb u uživatele – jako zdrojové porty

c) Dynamické (privátní)

- rozsah od **49152 ÷ 65535**
- jsou známé jako krátkodobé (ephemeral) porty, které se přidělují klientům při inicializaci spojení
- nejsou pevně přiděleny žádné aplikaci



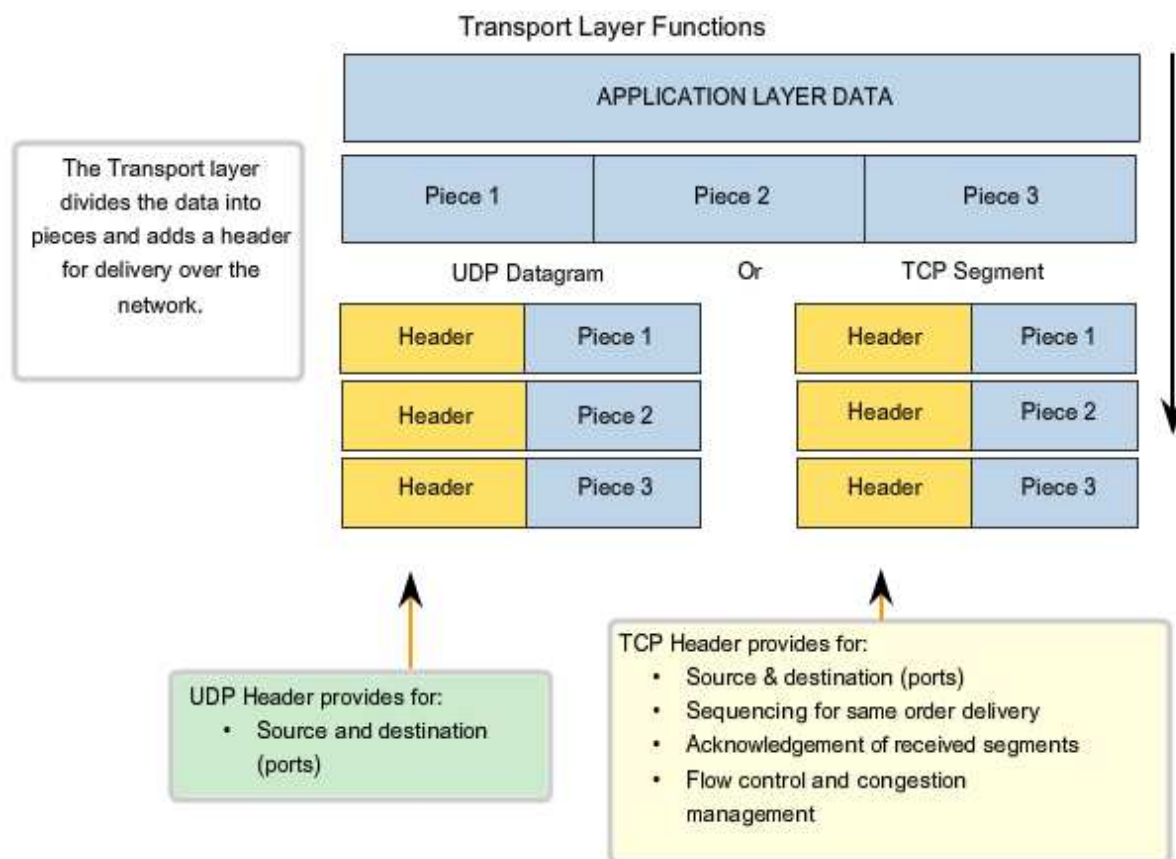
Některé aplikace mohou používat pro přenos oba protokoly – UDP i TCP – např. DNS – přičemž, oba pro označení používají stejné číslo portu 53.

Příkazem netstat lze zjistit množství navázaných TCP a UDP spojení na daném PC – lze tak zjistit zda na PC nejsou vytvořena nechtěná spojení, která mohou snižovat výkon PC.

4.1.6 Segmentace a znovu složení (Segmentation and Reassembly)

V případě, že by se nepoužívala segmentace, tak by se síť musely posílat velké datové celky – v době jejich odesílání by nemohly odesílat data jiné aplikace na stejném PC. Při vzniklé chybě by se musela celá zpráva znovu odeslat.

Segmentace – dělení dat na menší části – umožňuje používat multiplexování (sdružování a prokládání) dat z různých aplikací na společné médium.



V hlavičce TCP protokolu je pole *Sequence numer* (pořadové číslo), které umožňuje transportní vrstvě příjemce seřadit příchozí segmenty do správného pořadí. Hlavička UDP protokolu toto pole neobsahuje ⇒ transportní vrstva nedokáže zajisti předání jednotlivých částí dat do aplikační vrstvy ve správném pořadí.

Výhodou protokolu UDP je rychlejší přenos dat mezi aplikačními vrstvami – ale bez garance doručení všech dat a ve správném pořadí.

4.2 Protokol TCP

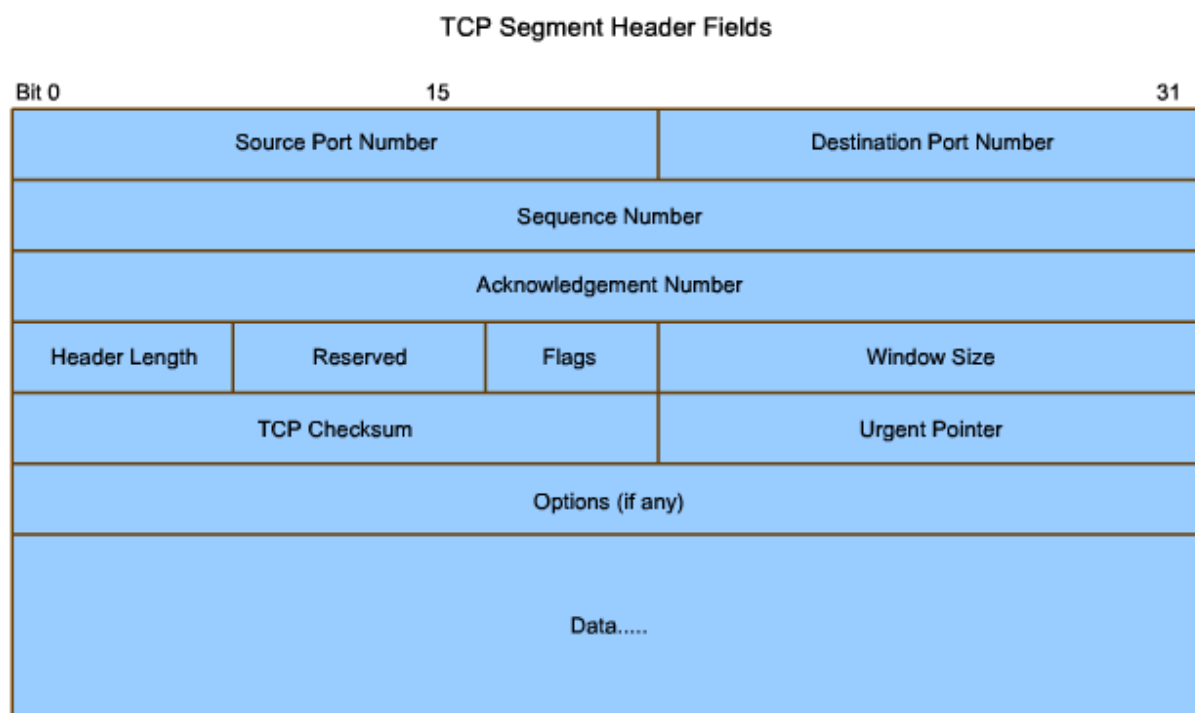
4.2.1 TCP – vytváří spolehlivou komunikaci

Hlavní rozdíl mezi protokoly TCP a UDP je spolehlivost (reliability). Spolehlivost u protokolu TCP je zajištěna prostřednictvím spojově orientovaného přenosu.

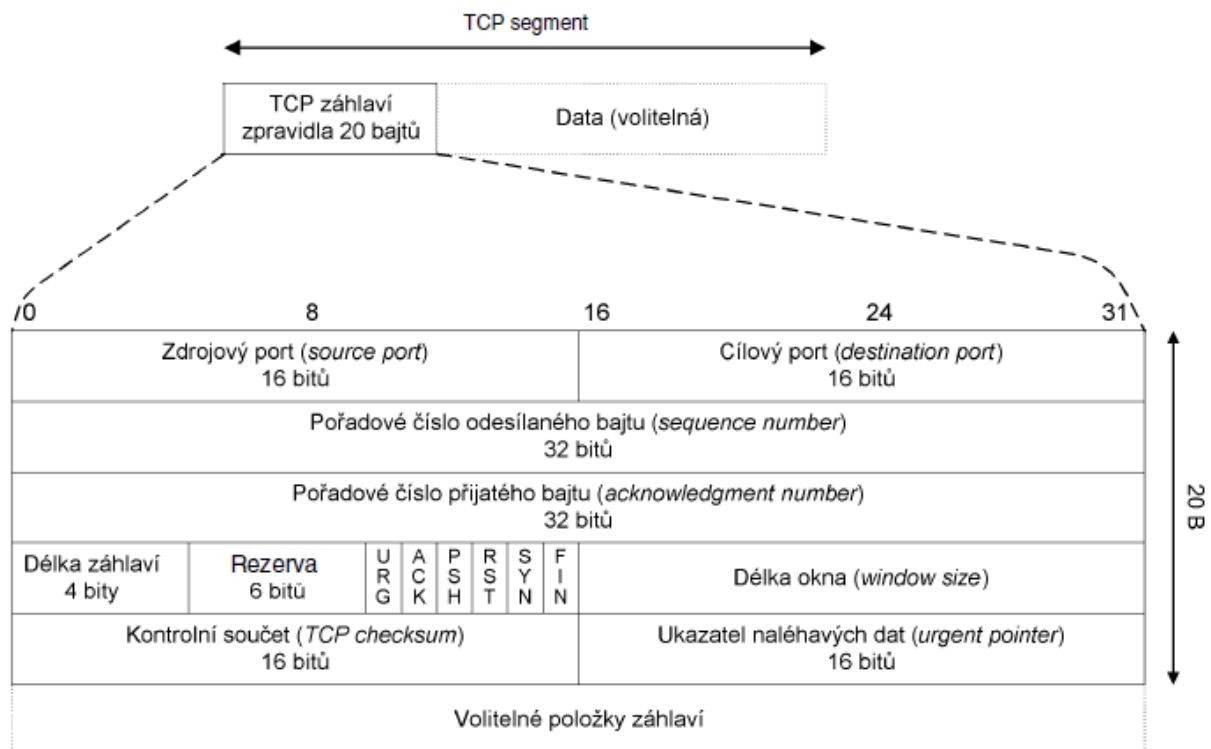
Před odesláním dat vytvoří transportní vrstvy spojení – PRO KAŽDÝ SMĚR PŘENOSU (každá z komunikujících stran pro svůj směr přenosu) – do takto vytvořeného spojení jsou pak vkládány očíslované segmenty, jejichž přijetí protější strana oznamuje odesílateli (acknowledgement). Pakliže odesílatel neobdrží do definované doby potvrzení, tak provede znovu odeslání nepotvrzených dat (retransmit).

Proces vytvoření spojení, potvrzovací mechanismus a znovu odesílání nepotvrzených segmentů zapříčiňuje odesílání většího množství dat přes síť $\Rightarrow$ což způsobuje nárůst dopravy v síti a z toho plynoucí větší zpoždění (delay).

Na druhou stranu ovšem vytvoření spojení mezi transportními vrstvami před začátkem přenosu aplikačních dat zajišťuje, že cílový host je schopen a připraven na komunikaci.



The fields of the TCP header enable TCP to provide connection-oriented, reliable data communications.



Význam jednotlivých částí TCP hlavičky:

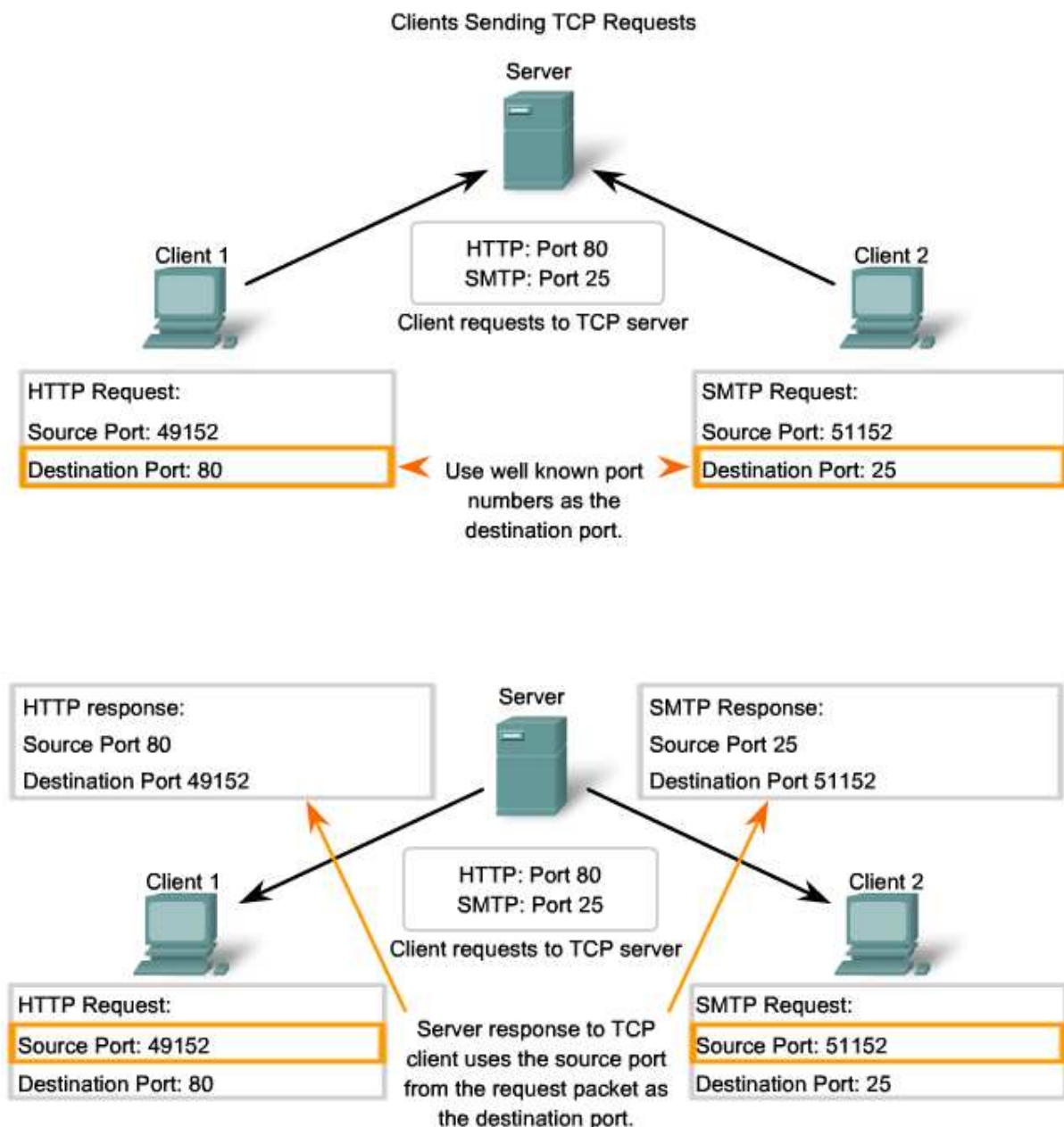
- **Zdrojový port (source port)** je port odesílatele TCP segmentu.
- **Cílový port (destination port)** je portem adresáta TCP segmentu – určuje, kterému procesu na cílovém uzlu jsou data určena.
- **Pořadové číslo odesílaného bajtu (sequence number)** je pořadové číslo prvního bajtu TCP segmentu v toku dat od odesílatele k příjemci (TCP segment nese bajty od **pořadového čísla odesílaného bajtu** až do délky segmentu). Tok dat v opačném směru má samostatné (jiné) číslování svých dat. Jelikož pořadové číslo odesílaného bajtu je 32 bitů dlouhé, tak po dosažení hodnoty $2^{32}-1$ nabude cyklicky opět hodnoty 0. Číslování obecně nezačíná od nuly (ani od nějaké určené konstanty), ale číslování začíná od náhodně zvoleného čísla. Vždy když je nastaven příznak SYN, tak operační systém odesílatele začíná znovu číslovat, tj. vygeneruje startovací pořadové číslo odesílaného bajtu, tzv. ISN (*Initial Sequence Number*).
- **Pořadové číslo přijatého bajtu (acknowledgment number)** vyjadřuje číslo následujícího bajtu, který je příjemce připraven přijmout, tj. příjemce potvrzuje, že správně přijal vše až do pořadového čísla přijatého bajtu mínus jedna. **Toto číslo je nastaveno na hodnotu, kterou odesílatel očekává v poli Sequence Number v následujícím paketu.**

- **Délka záhlaví** (*header length*) vyjadřuje délku záhlaví TCP segmentu v násobcích 32 bitů (4 bajtů) – podobně jako u IP-záhlaví.
- **Rezerva** (*reserved*) - toto 6ti bitové pole je rezervované a mělo by vždy být nulové.
- **Příznaky** (*flags*) – v poli příznaků mohou být nastaveny následující příznaky (jedná se o jednobitová pole):
 - **URG** – TCP segment nese naléhavá data
 - **ACK** – TCP segment má platné pole „Pořadové číslo přijatého bajtu“ (nastaven ve všech segmentech kromě prvního segmentu, kterým klient navazuje spojení)
 - **PSH** – zpravidla se používá k signalizaci, že TCP segment nese aplikační data, příjemce má tato data předávat aplikaci. Použití tohoto příznaku není ustáleno.
 - **RST** – odmítnutí TCP spojení
 - **SYN (Synchronize Sequence Number)** – odesílatel začíná s novou sekvencí číslování, tj. TCP segment nese pořadové číslo prvního odesílaného bajtu (ISN)
 - **FIN** – odesílatel ukončil odesílání dat. Přijetí TCP segmentu s příznakem FIN neznamena, že v opačném směru není dále možný přenos dat. Jelikož protokol TCP vytváří plně duplexní spojení, tak příznak FIN způsobí jen uzavření přenosu dat v jednom směru. V tomto směru už dále nebudou odesílány TCP segmenty obsahující příznak PSH.
- **Délka okna** (*window size*) vyjadřuje přírůstek pořadového čísla přijatého bajtu, který bude příjemcem ještě akceptován.
- **Kontrolní součet** (*TCP checksum*) kontrolní součet se počítá ze záhlaví TCP segmentu a z přenášených aplikačních dat.
- **Ukazatel naléhavých dat** (*urgent pointer*) může být nastaven pouze v případě, že je nastaven příznak URG. Přičte-li se tento ukazatel k pořadovému číslu odesílaného bajtu, pak ukazuje na konec úseku naléhavých dat. Odesílatel si přeje, aby příjemce tato naléhavá data přednostně zpracoval.

Pětice: zdrojový port, cílový port, zdrojová IP-adresa, cílová IP-adresa a protokol (TCP nebo UDP) jednoznačně identifikuje v daném okamžiku spojení v Internetu.

4.2.2 TCP – procesy na serveru

Server čeká (naslouchá) na přijetí požadavku od klienta $\Rightarrow$ klient je iniciátorem spojení. Na jednom serveru může běžet několik aplikací – každá aplikace má přiřazené jednoznačné číslo portu, na základě kterého jsou příchozí požadavky na server posílány příslušným serverovým procesům a aplikacím. Pakliže žádost obsahuje chybné číslo portu (cílového), tak tato žádost nebude serverem akceptována (nedojde k vytvoření spojení).



4.2.3 TCP – sestavení a ukončení spojení

Spojení se musí sestavit před samotným přenosem aplikačních dat – spojení se vytváří pro každý směr přenosu (vytváří se dvě spojení). Po ukončení přenosu aplikačních dat se spojení ukončí – opět pro každý směr přenosu. Vytvořením a ukončením přenosu se zajišťuje spolehlivost protokolu TCP.

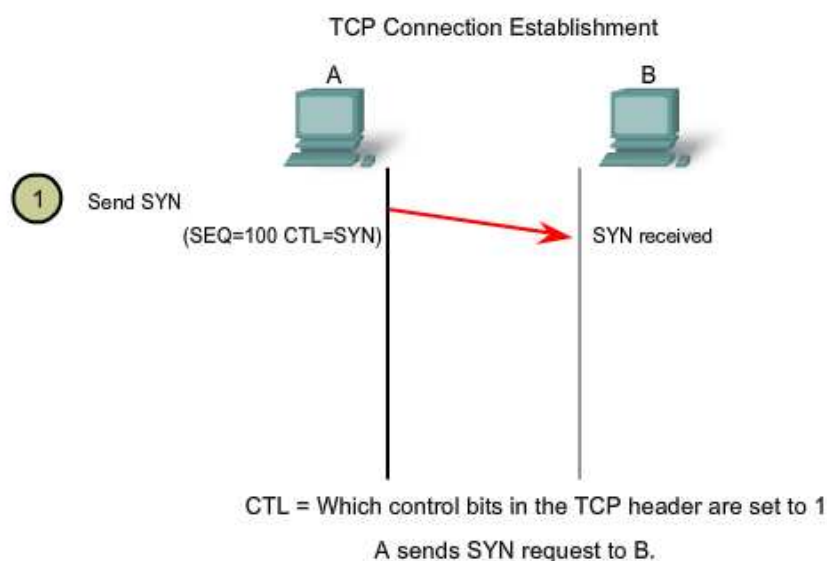
Jednotlivé komunikující strany si pomoci informací obsažených v hlavičce TCP protokolu vyměňují informace o tom, jaká data byly jednotlivými stranami úspěšně přijaty – dochází k potvrzování přijatých dat.

*Pro vytvoření spojení se používá mechanismus **THREE WAY HANDSHAKE** (třícestné podání ruky) – jednotlivé bity v poli Flags indikují v jakém stavu se nachází proces vytvoření spojení. **Three way handshake zajišťuje:***

- kontrolu, zda je cílové zařízení přítomné na síti
- ověření, zda má cílové zařízení aktivní požadovanou službu a zda je akceptován cílový port
- předání zdrojového portu (source port) klienta cílovému zařízení – server je informován o portu, který používá klient

Zřízení spojení (mezi klientem a serverem) probíhá ve třech krocích:

- 1) ***Klient (iniciátor spojení) odesílá segment, který v poli Sequence numer obsahuje ISN (Initial Sequence Numer – startovací pořadové číslo – náhodně zvolené 32-bitové číslo), od kterého začíná číslování odesílaných segmentů ve směru klient → server. V poli Flags je nastavený příznak SYN (Synchronize Sequence Numer). Tento odeslaný segment slouží jako žádost na sestavení spojení.***



TCP 3-way Handshake (SYN)

13	6.201109	192.168.254.254	10.1.1.1	DNS	Standard query r
14	6.202100	10.1.1.1	192.168.254.254	TCP	1069 > http [SYN
15	6.202513	192.168.254.254	10.1.1.1	TCP	http > 1069 [SYN
16	6.202543	10.1.1.1	192.168.254.254	TCP	1069 > http [ACK
17	6.202651	10.1.1.1	192.168.254.254	HTTP	GET / HTTP/1.1

⊕ Frame 14 (62 bytes on wire, 62 bytes captured)

⊕ Ethernet II, Src: QuantaCo_bd:0c:7c (00:c0:9f:bd:0c:7c), Dst: Cisco_cf:66:4c

⊕ Internet Protocol, Src: 10.1.1.1 (10.1.1.1), Dst: 192.168.254.254 (192.168.254.254)

⊖ Transmission Control Protocol, Src Port: 1069 (1069), Dst Port: http (80), Seq: 0, Win: 0, Len: 0

Source port: 1069 (1069)

Destination port: http (80)

Sequence number: 0 (relative sequence number)

Header length: 28 bytes

Flags: 0x02 (SYN)

0... .. = Congestion window Reduced (CWR): Not set

.0.. .. = ECN-Echo: Not set

..0. .. = Urgent: Not set

...0 .. = Acknowledgment: Not set

.... 0... = Push: Not set

.... .0.. = Reset: Not set

.... ..1. = Syn: Set

.... ...0 = Fin: Not set

window size: 65535

checksum: 0x0b0b [correct]

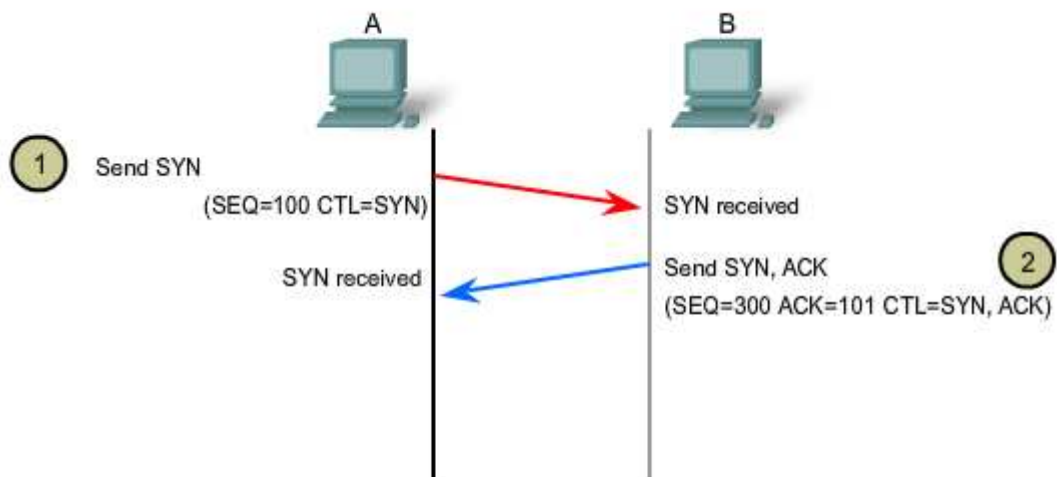
Protocol Analyzer shows initial client request for session in frame 14

TCP segment in this frame shows:

- SYN flag set to validate an initial Sequence number
- Randomized sequence number valid (relative value is 0)
- Random source port 1069
- Well known destination port is 80 (HTTP port) indicates web server (httpd)

- 2) Server odpovídá odesláním segmentu, který v poli **Pořadové číslo přijatého bajtu** (**acknowledgment numer**) obsahuje hodnotu čísla sequence numer ze segmentu přijatého od klienta zvětšené o jedničku. Zároveň server odesílá v poli **Pořadové číslo odesílaného bajtu** (**sequence number**) vlastní náhodně vygenerované ISN ((**Initial Sequence Numer** – startovací pořadové číslo), od kterého budou číslovány segmenty posílané ve směru server → klient. V poli **Flags** jsou nastavené příznaky **SYN** (**Synchronize Sequence Numer**) a **ACK** (**Acknowledgment Numer**). Příznak **ACK** už zůstává nastavený po celou dobu trvání spojení a symbolizuje, že segment má platné pole **Pořadové číslo přijatého bajtu** – toto číslo je nastaveno na hodnotu, kterou odesílatel očekává v poli **Sequence Number** v následujícím paketu.

TCP Connection Establishment



CTL = Which control bits in the TCP header are set to 1

B sends ACK response and SYN request to A.

TCP 3-way Handshake (SYN, ACK)

13	6.201109	192.168.254.254	10.1.1.1	DNS	Standard query
14	6.202100	10.1.1.1	192.168.254.254	TCP	1069 > http [S]
15	6.202513	192.168.254.254	10.1.1.1	TCP	http > 1069 [S]
16	6.202543	10.1.1.1	192.168.254.254	TCP	1069 > http [A]
17	6.202651	10.1.1.1	192.168.254.254	HTTP	GET / HTTP/1.1

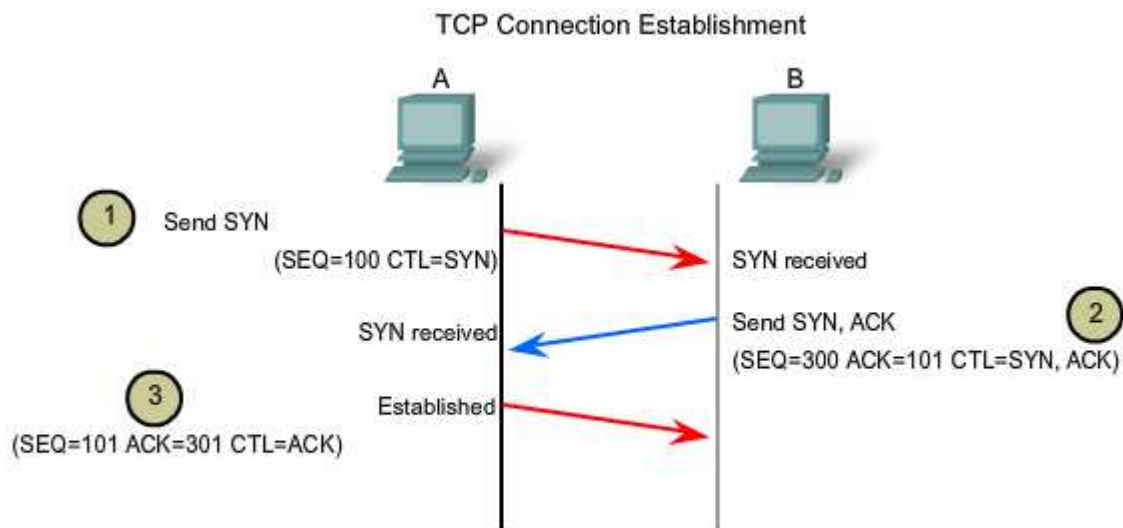
```

+ Frame 15 (62 bytes on wire (8 bytes captured) on interface 0)
+ Ethernet II, Src: Cisco_cf:66:40 (00:0c:85:cf:66:40), Dst: QuantaCo_bd:0c:
+ Internet Protocol, Src: 192.168.254.254 (192.168.254.254), Dst: 10.1.1.1 (
+ Transmission Control Protocol, Src Port: http (80), Dst Port: 1069 (1069),
  Source port: http (80)
  Destination port: 1069 (1069)
  Sequence number: 0 (relative sequence number)
  Acknowledgement number: 1 (relative ack number)
  Header length: 28 bytes
  Flags: 0x12 (SYN, ACK)
    0... .... = Congestion window Reduced (CWR): Not set
    .0.. .... = ECN-Echo: Not set
    ..0. .... = Urgent: Not set
    ...1 .... = Acknowledgment: Set
    .... 0... = Push: Not set
    .... .0.. = Reset: Not set
    .... ..1. = Syn: Set
    .... ...0 = Fin: Not set
  Window size: 5840
  Checksum: 0x91a4 [correct]
  
```

A protocol analyzer shows server response in frame 15

- ACK flag set to indicate a valid Acknowledgement number
- Acknowledgement number response to initial sequence number as relative value of 1
- SYN flag set to indicate the Initial sequence number for the server to client session
- Destination port number of 1069 to corresponding to the clients source port
- Source port number of 80 (HTTP) indicating the web server service (httpd)

- 3) Klient odpovídá odesláním segmentu, který v poli **Pořadové číslo přijatého bajtu** (*acknowledgment number*) obsahuje hodnotu čísla *sequence number* ze segmentu přijatého od serveru zvětšené o jedničku. Zároveň klient odesílá v poli **Pořadové číslo odesílaného bajtu** (*sequence number*) vlastní náhodně vygenerované ISN ((*Initial Sequence Numer* – startovací pořadové číslo) zvětšené o jedničku. V poli **Flags** je nastavený příznak **ACK** (*Acknowledgment Numer*).



CTL = Which control bits in the TCP header are set to 1

A sends ACK response to B.

TCP 3-way Handshake (ACK)

13	6.201109	192.168.254.254	10.1.1.1	DNS	Standard query re
14	6.202100	10.1.1.1	192.168.254.254	TCP	1069 > http [SYN]
15	6.202513	192.168.254.254	10.1.1.1	TCP	http > 1069 [SYN,
16	6.202543	10.1.1.1	192.168.254.254	TCP	1069 > http [ACK]
17	6.202651	10.1.1.1	192.168.254.254	HTTP	GET / HTTP/1.1

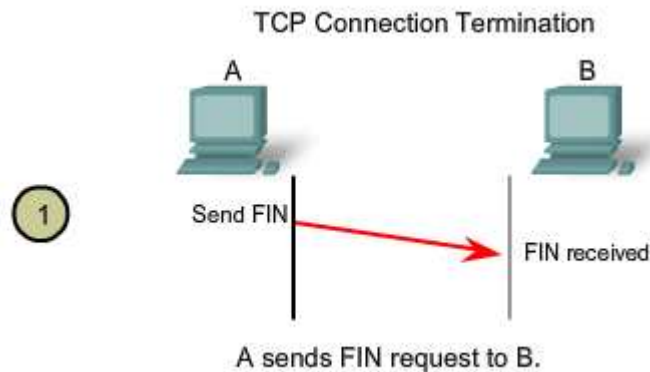
+ Frame 16 (54 bytes on wire, 54 bytes captured)	
+ Ethernet II, Src: QuantaCo_bd:0c:7c (00:c0:9f:bd:0c:7c), Dst: Cisco_cf:66:40	
+ Internet Protocol, Src: 10.1.1.1 (10.1.1.1), Dst: 192.168.254.254 (192.168.254.254)	
+ Transmission Control Protocol, Src Port: 1069 (1069), Dst Port: http (80), Seq	
Source port: 1069 (1069)	
Destination port: http (80)	
Sequence number: 1 (relative sequence number)	
Acknowledgement number: 1 (relative ack number)	
Header length: 20 bytes	
+ Flags: 0x10 (ACK)	
0... .. = Congestion window Reduced (CWR): Not set	
.0.. = ECN-Echo: Not set	
..0. = Urgent: Not set	
...1 = Acknowledgment: Set	
.... 0... = Push: Not set	
.... .0.. = Reset: Not set	
.... ..0. = Syn: Not set	
.... ...0 = Fin: Not set	
window size: 65535	
Checksum: 0xd538 [correct]	

Ukončení TCP spojení

Pro ukončení spojení (pro každý jeho směr) se používá TWO WAY HANDSHAKE – v poli Flags je nastaven příznak FIN (Finish), který musí být druhou stranou potvrzen (ACK).

Mezi klientem a serverem se pro ukončení obou směrů spojení posílají čtyři segmenty:

- 1) Když klient nemá k odeslání další data, odešle segment, který má v poli Flags nastaven příznak FIN.

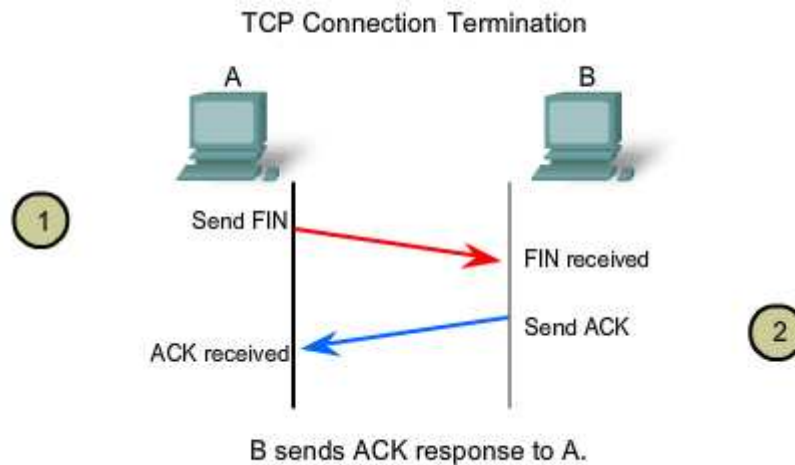


TCP Session Termination (FIN)

20	6.203876	192.168.254.254	10.1.1.1	TCP	http > 1069 [FIN,
21	6.203899	10.1.1.1	192.168.254.254	TCP	1069 > http [ACK]
22	6.204139	10.1.1.1	192.168.254.254	TCP	1069 > http [FIN,
23	6.204416	192.168.254.254	10.1.1.1	TCP	http > 1069 [ACK]
24	6.204662	10.1.1.1	192.168.254.254	HTTP	Standard query v. 0

Frame 20 (60 bytes on wire, 60 bytes captured)	
Ethernet II, Src: Cisco_cf:66:40 (00:0c:85:cf:66:40), Dst: Quantaco_bd:0c:7c	
Internet Protocol, Src: 192.168.254.254 (192.168.254.254), Dst: 10.1.1.1 (10.1.1.1)	
Transmission Control Protocol, Src Port: http (80), Dst Port: 1069 (1069), Seq	
Source port: http (80)	
Destination port: 1069 (1069)	
Sequence number: 440 (relative sequence number)	
Acknowledgement number: 414 (relative ack number)	
Header length: 20 bytes	
Flags: 0x11 (FIN, ACK)	
0... .. = Congestion window reduced (CWR): Not set	
.0.. = ECN-Echo: Not set	
..0. = Urgent: Not set	
...1 = Acknowledgment: set	
.... 0... = Push: Not set	
.... .0.. = Reset: Not set	
.... ..0. = Syn: Not set	
.... ...1 = Fin: Set	
Window size: 6432	
Checksum: 0xb8c3 [correct]	

- 2) Server odpovídá odesláním potvrzení ACK přijetí segmentu s příznakem FIN – tím je ukončen jeden směr přenosu – směr klient → server.



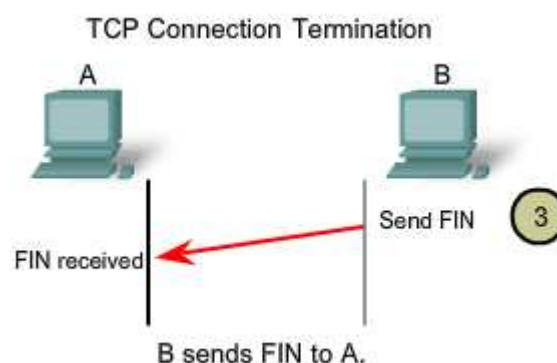
TCP Session Termination (ACK)

20	6.203876	192.168.254.254	10.1.1.1	TCP	http > 1069 [FIN,
21	6.203899	10.1.1.1	192.168.254.254	TCP	1069 > http [ACK]
22	6.204139	10.1.1.1	192.168.254.254	TCP	1069 > http [FIN,
23	6.204416	192.168.254.254	10.1.1.1	TCP	http > 1069 [ACK]
24	6.602668	10.1.1.1	192.168.254.254	DNS	Standard query A

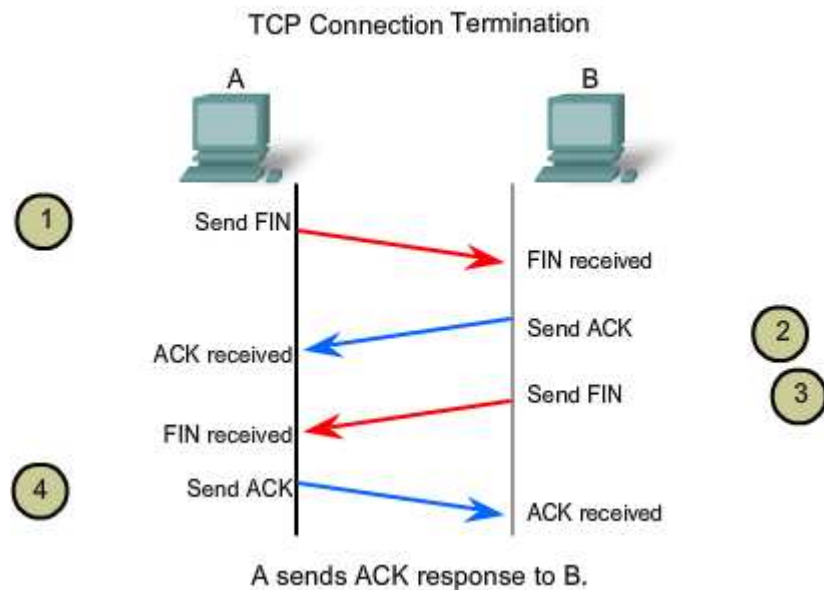

```

+ Frame 21 (54 bytes on wire (42 bytes captured) on interface 0)
+ Ethernet II, Src: QuantaCo_bd:0c:7c (00:c0:9f:bd:0c:7c), Dst: Cisco_cf:66:40
+ Internet Protocol, Src: 10.1.1.1 (10.1.1.1), Dst: 192.168.254.254 (192.168.254.254)
+ Transmission Control Protocol, Src Port: 1069 (1069), Dst Port: http (80), Seq: 414, Win: 0, Len: 0
  Source port: 1069 (1069)
  Destination port: http (80)
  Sequence number: 414 (relative sequence number)
  Acknowledgement number: 441 (relative ack number)
  Header length: 20 bytes
  Flags: 0x10 (ACK)
    0... .... = Congestion window Reduced (CWR): Not set
    .0.. .... = ECN-Echo: Not set
    ..0. .... = Urgent: Not set
    ...1 .... = Acknowledgment: Set
    .... 0... = Push: Not set
    .... .0.. = Reset: Not set
    .... ..0. = Syn: Not set
    .... ...0 = Fin: Not set
  Window size: 65096
  Checksum: 0xd39a [correct]
  
```

- 3) Když server nemá k odeslání další data – odešle segment, který má v poli Flags nastaven příznak FIN.

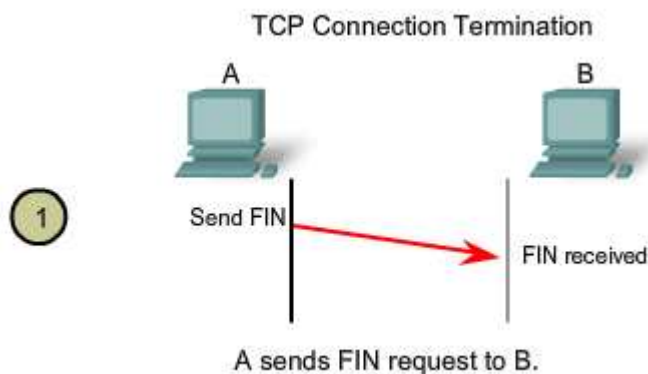


- 4) Klient odpovídá odesláním potvrzení ACK přijetí segmentu s příznakem FIN – tím je ukončen druhý směr přenosu – směr server → klient.

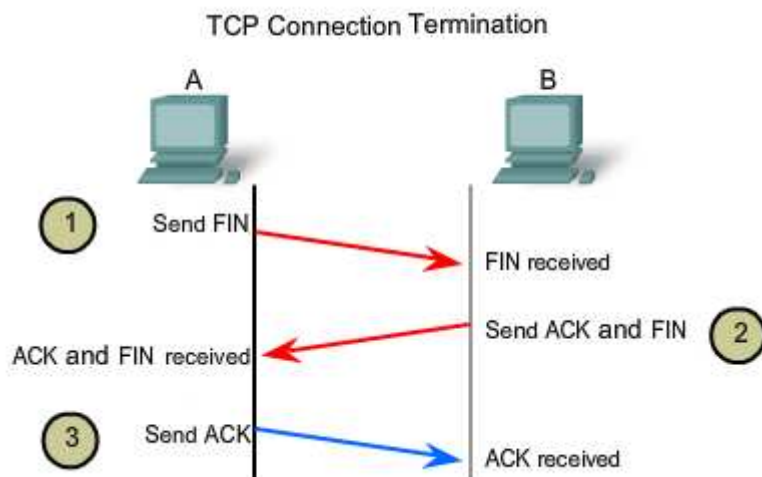


Spojení lze ukončit také pomocí THREE WAY HANDSHAKE – ukončení spojení bude provedeno ve třech krocích:

- 1) Když klient nemá k odeslání další data, odešle segment, který má v poli Flags nastaven příznak FIN.



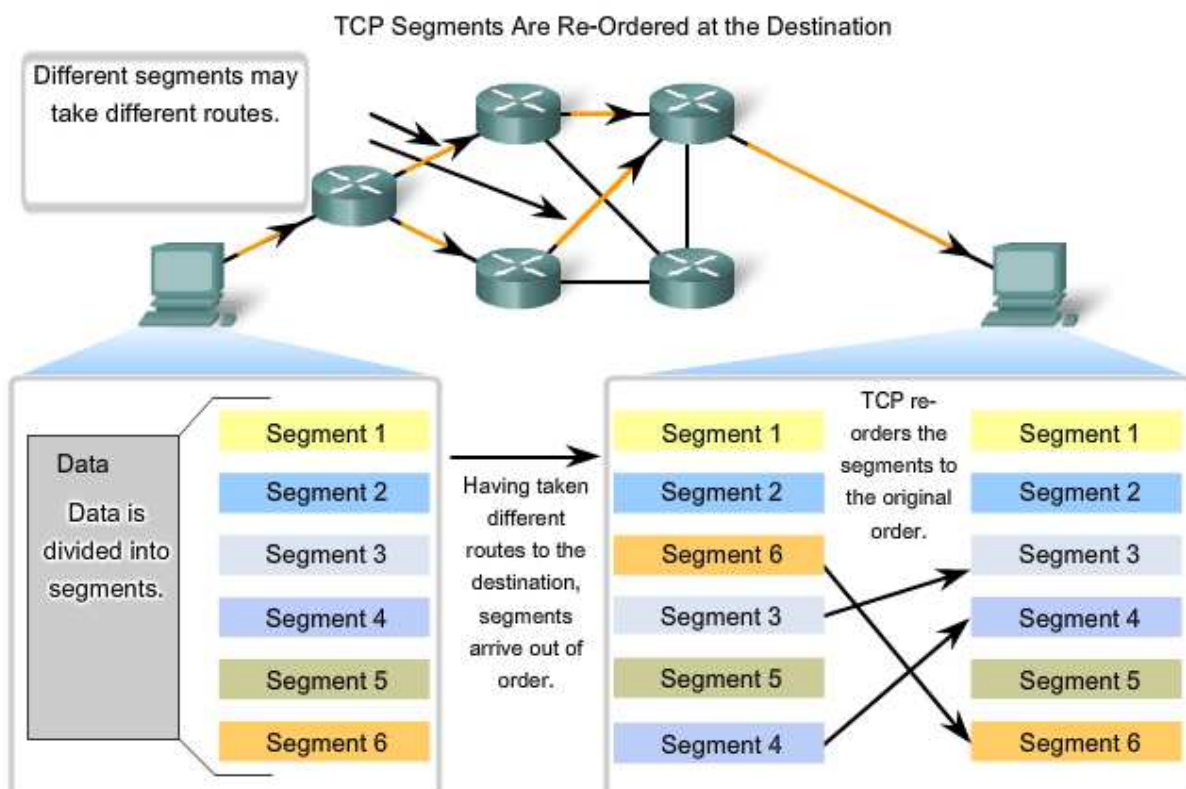
- 2) Když ani server nemá k odeslání další data – odešle segment, kterým potvrdí (ACK) přijetí segmentu s příznakem FIN – došlo k ukončení spojení ve směru klient → server. Současně bude v odesílaném segmentu nastaven v poli Flags příznak FIN pro ukončení spojení ve směru server → klient.
- 3) Klient odpoví odesláním potvrzení ACK přijetí segmentu s příznakem FIN – tím je ukončen i druhý směr přenosu – směr server → klient.



4.3 Vytvoření TCP relací

4.3.1 Znovu sestavení TCP segmentů (Segment reassembly)

Když se posílají data použitím TCP protokolu, tak může nastat situace, že jednotlivé odeslané segmenty dorazí do cíle v nesprávném pořadí – každý může putovat jinou cestou – viz obrázek.

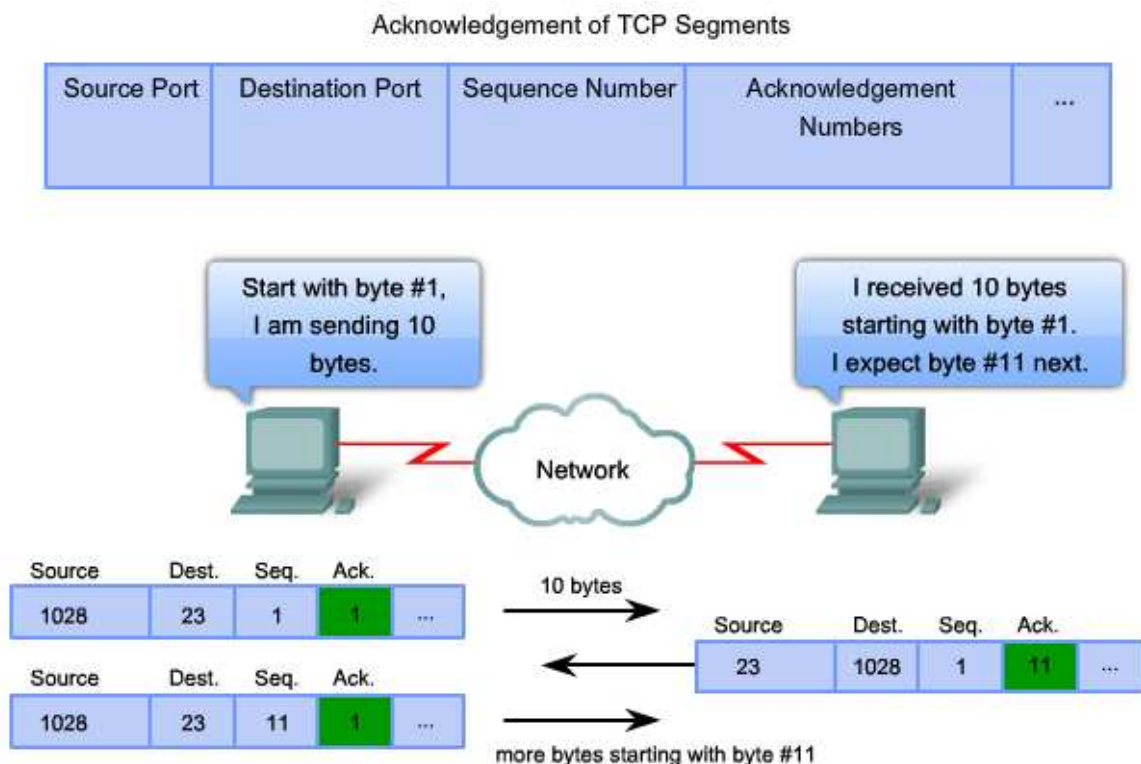


Pro jejich seřazení do správného pořadí slouží pole SN (Sequence Number). Při sestavování TCP spojení se SN nastaví na hodnotu ISN (Initial Sequence Number– startovací pořadové číslo) a k této hodnotě se pak přičítá počet odeslaných bajtů v jednotlivých segmentech. Příchozí segmenty jsou u příjemce řazeny do přijímacího bufferu (receiving buffer) a na základě pořadového čísla SN jsou seřazeny do správného pořadí a poté předány do aplikační vrstvy – podle čísla portu.

4.3.2 TCP potvrzování (TCP acknowledgment)

Jednou z funkcí TCP protokolu je zajištění, že veškeré odeslané segmenty dorazí do cíle. Každý odeslaný segment obsahuje v hlavičce pole **SN (Sequence Number)** – což je pořadové číslo prvního bajtu v daném segmentu.

Příjemce potvrzuje přijetí segmentu pomocí pole Acknowledgment number – což je číslo následujícího očekávaného bajtu $\Rightarrow$ příjemce potvrzuje správné přijetí všech bajtů do hodnoty tohoto pole minus 1. Hodnota pole Acknowledgment number je pak obsažena v poli SN následujícího odeslaného segmentu – viz obrázek.



Aby se nemuselo potvrzovat přijetí každého segmentu zvlášť – což by způsobovalo nárůst přenášených řídicích dat přes síť – existuje možnost odeslat několik segmentů, bez čekání na jejich potvrzení. Množství odeslaných dat v segmentech (bez nutnosti potvrzení) je dáno hodnotou pole **WINDOWS SIZE.**

4.3.3 Znovu odeslání TCP segmentu (TCP retransmission)

Bez ohledu na to, jak dobře je datová síť navržena, dochází v ní ke ztrátě dat – **protokol TCP poskytuje možnost zjištění ztráty přenášených dat (jejich neúplnost) a jejich znovu odeslání – realizuje se to pomocí nepotvrzení nedoručených dat.**

Cílový host potvrzuje pouze přijetí dat, které tvoří souvislý celek – např. jsou-li přijaty segmenty s pořadovými čísly 1500 ÷ 3000 a 3400 ÷ 3500 bude potvrzovací číslo přijatého bajtu (Acknowledgment Number) nastaveno na hodnotu 3001.

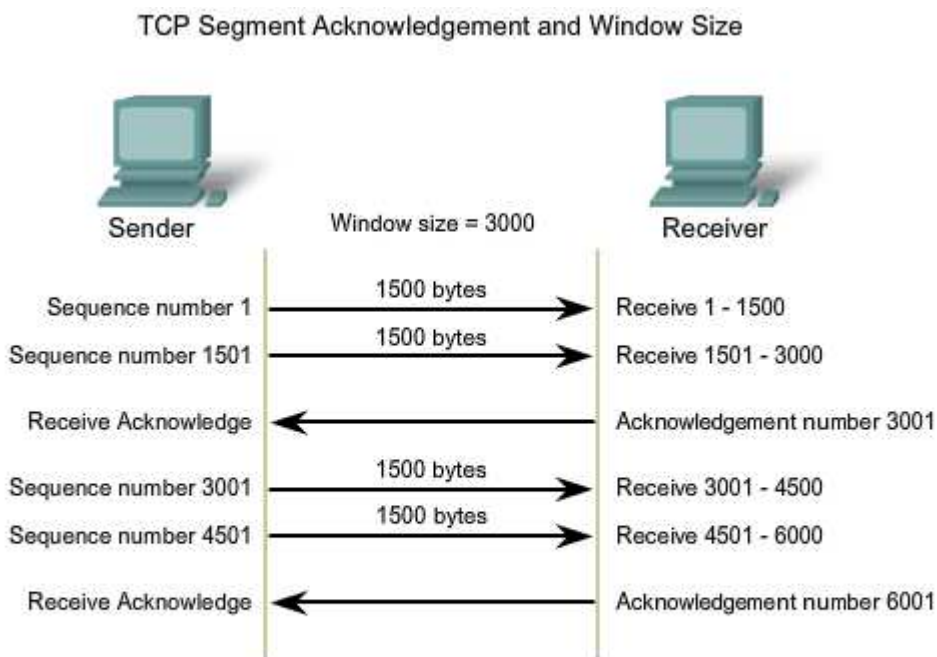
Pokud odesílatel neobdrží potvrzení do předdefinované doby, tak opětovně odešle veškerá data, která nebyla příjemcem potvrzena. Při odesílání se kopie odeslaných segmentů ukládají do paměti a spouští se časovač – pakliže potvrzení nedorazí do předdefinované doby jsou segmenty opětovně odeslány.

Existuje také možnost tzv. selektivního potvrzování (selective acknowledgment) ⇒ příjemce může potvrdit přijetí segmentů, jejichž pořadové číslo netvoří souvislou řadu ⇒ odesílatel znovu odešle pouze ty segmenty, které nebyly potvrzeny.

4.3.4 Ochrana proti zahlcení – minimalizace ztrát segmentů

Řízení toku (Flow kontrol)

Pomocí řízení toku se zabráňuje přetížení (zahlcení – congestion) sítě a zahlcení cílového hosta – což by vedlo ke ztrátě segmentů a k nutnosti jejich znovu odeslání.



The **window size** determines the number of bytes sent before an acknowledgment is expected.
The **acknowledgement** number is the number of the next expected byte.

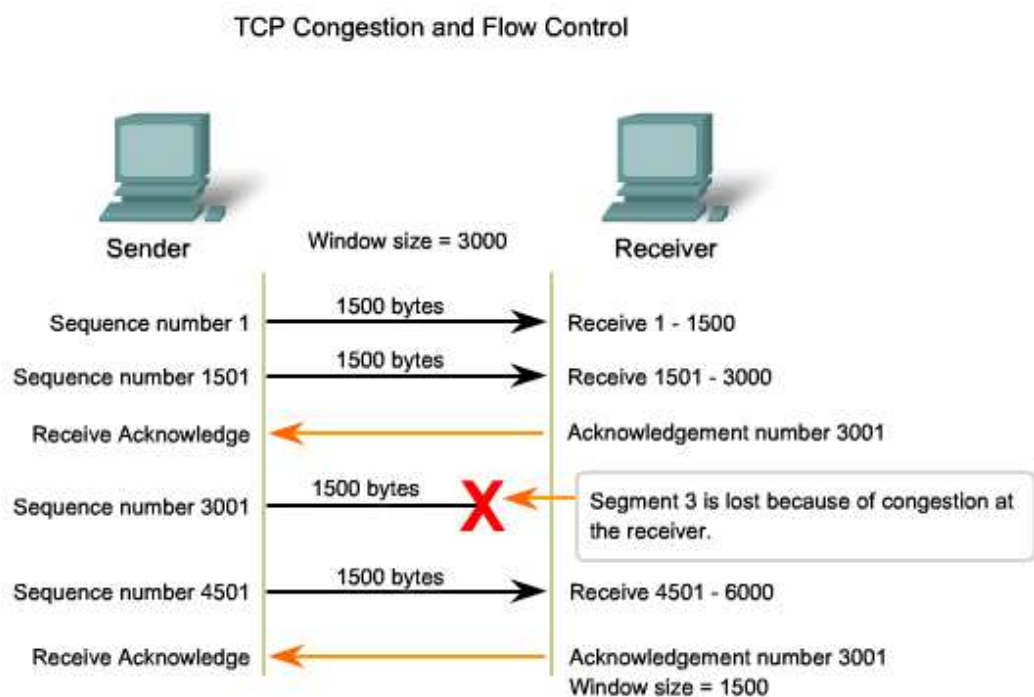
Řízení toku je řešeno pomocí pole **Window size (velikost okna)** – jehož hodnota udává množství dat, které mohou být odeslány před přijetím potvrzení. Výchozí hodnota window size (initial windows size) je určena v průběhu navázání spojení (three way handshake).

Na obrázku výše je hodnota window size nastavena na 3000 $\Rightarrow$ jakmile odesílatel odešle 3000 bajtů, tak musí počkat na přijetí potvrzení a pak teprve může odesílat další segmenty pro toto spojení.

Zmenšení velikosti okna (Reducing window size)

Když dojde k zahlcení sítě nebo cílového hosta, lze dynamicky změnit (zmenšit) hodnotu velikosti okna (Windows size). Což má za následek snížení množství přenášovaných dat přes síť – protože odesílatel bude častěji čekat na přijetí potvrzení odeslaných dat.

Velikost hodnoty Windows size určuje příjemce – pakliže nestačí zpracovávat příchozí segmenty a dochází k zaplnění vyrovnávací paměti (buffer) – tak jako součást potvrzení dat pošle zmenšenou hodnotu window size.



If segments are lost because of congestion, the Receiver will acknowledge the last received sequential segment and reply with a reduced window size.

Na příkladu je vidět, že došlo z důvodu zahlcení (congestion) příjemce ke ztrátě jednoho segmentu. Příjemce snížil hodnotu window size ze 3000 na 1500. **Pakliže nedojde po určitou dobu k opětovné ztrátě segmentu, může příjemce opětovně navýšit hodnotu window size. K navyšování hodnoty window size dochází do té doby, než dojde k další ztrátě segmentu, která vede k opětovnému snížení hodnoty velikosti okna.**

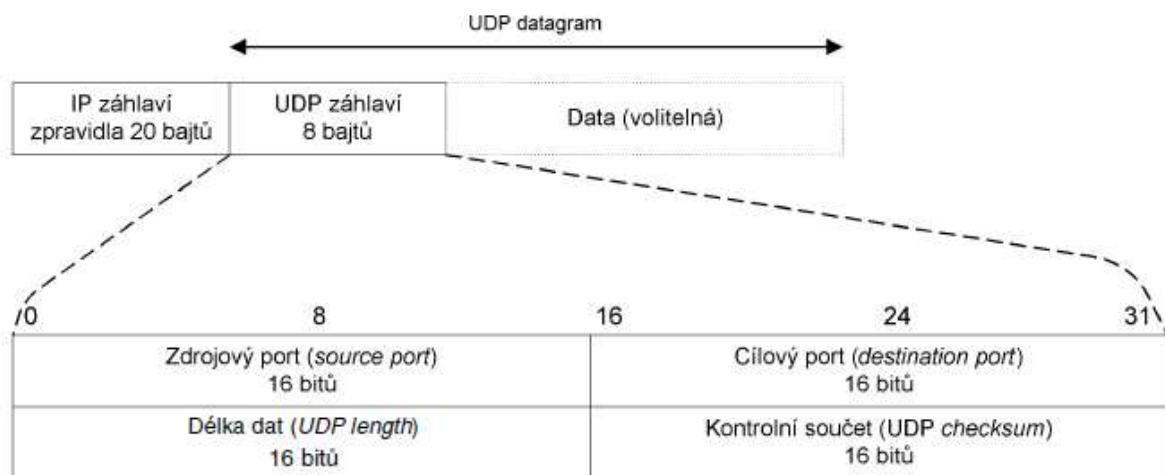
4.4 Protokol UDP

4.4.1 UDP – nižším množstvím řídicích informací versus spolehlivost

Protokol UDP (User Datagram Protocol) je jednoduchou alternativou protokolu TCP.

Protokol UDP je nespojovaná služba (na rozdíl od protokolu TCP), **tj. nenavazuje spojení.**

Odesílatel odešle UDP datagram příjemci a už se nestará o to, zdali se datagram náhodou neztratil. **UDP datagramy jsou baleny do IP-datagramu**, jak je znázorněno na obrázku



Záhlaví UDP protokolu je tvořeno 8 bajty a obsahuje:

- **číslo zdrojového a cílového portu** (source port, destination port) – což je zcela analogické protokolu TCP. Čísla portů protokolu UDP nesouvisí s čísly portů protokolu TCP – protokol UDP má svou nezávislou sadu čísel portů
- **délku dat** (UDP length) – pole obsahuje délku UDP datagramu (délku záhlaví + délku dat)
- **kontrolní součet** (UDP checksum) – toto pole nemusí být povinně vyplněné ⇒ výpočet kontrolního součtu je v protokolu UDP nepovinný

Mezi aplikační protokoly, které používají transportní protokol UDP například patří:

- DNS (Domain Name System)
- DHCP (Dynamic Host Configuration Protocol)
- RIP (Routing Information Protocol)
- TFTP (Trivial File Transfer Protocol)

Aplikace využívající UDP protokol upřednostňují přenos dat s menším zpožděním na úkor možným ztrátám dat při přenosu ⇒ částečné ztráty dat nemají na kvalitu komunikace takový dopad jako zpoždění – např. u IP telefonie.

4.4.2 UDP – znovu složení datagramu

UDP je nespojově orientovaný protokol $\Rightarrow$ před přenosem dat nevytváří spojení – data jsou odesílána bezprostředně po vytvoření v aplikační vrstvě.

Datový celek se u protokolu UDP nazývá DATAGRAM – neobsahuje pořadové číslo (sequence number), což má za následek:

- na přijímací straně nelze příchozí datagramu seřadit do správného pořadí
- nelze detekovat ztracené datagramy, ani je znovu odeslat

